



VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan , Sehore

Madhya Pradesh - 466114

CSA1021 INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING

REG.NO: 26BCE10876

NAME : DEEPANWITA GERA

BRANCH: CSE Core

SEMESTER: Fall Semester 2026-27

1. PROBLEM STATEMENT :

In a college canteen, managing food orders, calculating bills, maintaining stock, and handling payments manually can be time-consuming and may lead to calculation errors. During busy hours, manual order management can also result in incorrect quantities, incorrect billing, and difficulty in maintaining inventory records.

The **Smart Canteen Ordering and Billing System** is developed to computerize the basic canteen operations. The system allows customers to view food items, search for food, place orders, calculate bills automatically, select a payment method, and receive a generated bill.

The system also provides an administrator section through which the administrator can view inventory, restock food items, update prices, add new food items, and remove existing food items.

The project is implemented using Python and demonstrates programming concepts such as functions, loops, conditional statements, dictionaries, exception handling, file handling, and date-time operations.

2. OBJECTIVES :

The main objectives of the Smart Canteen Ordering and Billing System are:

1. To develop a computerized canteen ordering system using Python.
2. To allow customers to view the available food menu.
3. To allow customers to search and select food items.
4. To accept the quantity of food items required by the customer.
5. To check the availability of food items using inventory data.
6. To automatically calculate the subtotal of an order.
7. To automatically calculate GST on the order.
8. To apply discounts when the order satisfies the required condition.
9. To calculate the final bill automatically.
10. To provide Cash, UPI, and Card payment options.
11. To generate a unique order ID for each completed order.
12. To save generated bills in a text file.
13. To automatically update inventory after an order is completed.
14. To provide an admin panel for inventory management.
15. To reduce manual calculation and human errors.
16. To demonstrate practical application of Python programming concepts.

3. FUNCTIONAL REQUIREMENTS :

Functional requirements describe what the system should be able to do.

3.1 Customer Management

The system should:

- Accept the customer's name.
- Display the customer menu.
- Allow the customer to perform different operations.
- Allow the customer to return to the main menu.

3.2 Food Menu Management

The system should:

- Display all available food items.
- Display food name, category, price, and stock.
- Display food categories.
- Allow customers to search for food items.

3.3 Order Management

The system should:

- Allow customers to add food items.
- Accept the required quantity.
- Check whether the selected item exists.
- Check whether sufficient stock is available.
- Display the current order.
- Remove selected items from the order.
- Clear the complete order when required.

3.4 Billing

The system should:

- Calculate the subtotal.
- Calculate GST.
- Apply discount when applicable.
- Calculate the final payable amount.
- Generate an order ID.
- Display customer and order details.

The billing formula used is:

Subtotal = Σ (Price $\times$ Quantity)

GST = Subtotal $\times$ 5%

If:

Subtotal $\geq$ ₹500

then:

Discount = Subtotal $\times$ 10%

Otherwise:

Discount = ₹0

Finally:

Final Amount = Subtotal + GST – Discount

3.5 Payment

The system should provide:

- Cash payment
- UPI payment
- Card payment

The current project simulates payment selection and does not connect to a real banking or payment gateway.

3.6 Bill Storage

After checkout, the system should save the bill in:
canteen_bills.txt

The file stores information such as:

- Order ID
- Customer name
- Ordered items
- Subtotal
- GST
- Discount
- Final amount
- Payment method
- Date and time

3.7 Inventory Management

The system should:

- Display current stock.
- Reduce stock after successful checkout.
- Allow the administrator to restock items.
- Allow the administrator to add new food items.
- Allow the administrator to remove food items.

3.8 Price Management

The administrator should be able to update the price of an existing food item.

3.9 Admin Authentication

The system provides an admin login using a username and password.

For the educational version:

Username: admin

Password: 1234

4. NON-FUNCTIONAL REQUIREMENTS :

Non-functional requirements describe the quality and operational characteristics of the system.

4.1 Usability

The system should have simple menus and clear instructions so that a first-time user can operate it easily.

4.2 Reliability

The system should perform calculations consistently and prevent invalid input from crashing the program.

4.3 Accuracy

The system should accurately calculate:

- Food cost
- Subtotal
- GST
- Discount
- Final amount

4.4 Performance

The system should respond quickly because all calculations and menu operations are performed locally.

4.5 Maintainability

The program is divided into separate functions such as:

- display_menu()
- add_item()
- view_order()
- checkout()
- generate_bill()
- admin_menu()

This makes the program easier to understand and modify.

4.6 Security

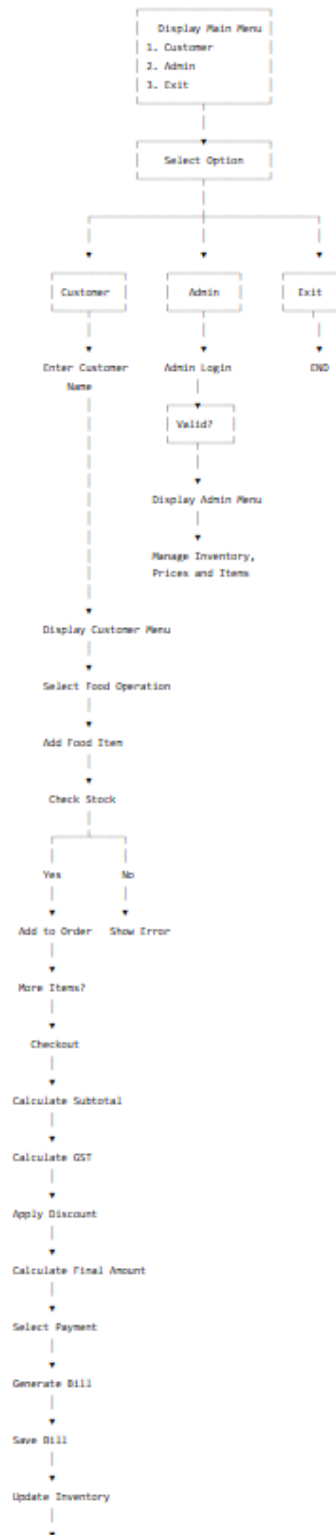
The system provides basic administrator authentication. However, because this is an educational Python project, the username and password are stored directly in the program. A real-world system should use secure password storage and authentication.

4.7 Portability

The program can run on systems that have Python installed.

5. SYSTEM ARCHITECTURE DIAGRAM

The system follows a simple layered architecture.



Explanation :

The **User Layer** contains the customer and administrator.

The **Application Layer** handles menus, ordering, billing, payment, and inventory operations.

The **Logic/Data Layer** stores menu and order information using Python dictionaries and performs calculations.

The **Storage Layer** stores completed bills in the canteen_bills.txt text file.

6. DATABASE / STORAGE DESIGN :

6.1 Storage Used

The project does not use a traditional database such as MySQL or SQLite. Instead, it uses:

In-memory storage

Python dictionaries are used to store:

- Food menu
- Food category
- Food price
- Food stock
- Customer order

Permanent storage

Completed bills are stored in:
canteen_bills.txt

6.2 LOGICAL DATA MODEL

MENU

Field	Description
Item ID	Unique number of food item
Name	Name of food
Category	Food category
Price	Price of food
Stock	Available quantity

Example:

MENU

Item ID	Name	Price	Stock
1	Veg Burger	₹60	20
2	Cheese Burger	₹80	15
3	Veg Pizza	₹120	12
4	Samosa	₹20	40

6.3 ORDER STORAGE :

The customer's order is stored using a Python dictionary.

ORDER

Item ID → Quantity

1 → 2
5 → 1
13 → 3

This means the customer has ordered:

- 2 Veg Burgers
- 1 French Fries
- 3 Teas

6.4 BILL STORAGE

The completed bill contains:

Order ID

Customer Name

Food Items

Quantity

Price

Subtotal

GST

Discount

Final Amount

Payment Method

Date and Time

The information is saved into:

canteen_bills.txt

The file is opened using append mode so that previous bills are not overwritten.

7. SCHEMA DESIGN :

Since the project uses dictionaries instead of a relational database, the equivalent schema can be represented as follows.

Food Item Schema

Food Item:

item_id : Integer

name : String

category : String

price : Float/Integer

stock : Integer

Order Schema

Order:

item_id : Integer

quantity : Integer

Bill Schema

Bill:

order_id : Integer

customer_name : String

items : List

subtotal : Float

gst : Float

discount : Float

final_amount : Float

payment_method : String

date_time : DateTime

8. ALGORITHM :

Step 1-

Start the program.

Step 2-

Display the main menu:

1. Customer
2. Admin
3. Exit

Step 3-

If Customer is selected, ask for the customer's name.

Step 4-

Display the customer menu.

Step 5-

Allow the customer to view, search, add, remove, or clear food items.

Step 6-

When adding an item, check whether the item exists and whether sufficient stock is available.

Step 7-

Store the selected item and quantity in the order dictionary.

Step 8-

Calculate subtotal using:

$$\text{Subtotal} = \Sigma(\text{Price} \times \text{Quantity})$$

Step 9-

Calculate GST:

$$\text{GST} = \text{Subtotal} \times 0.05$$

Step 10-

If subtotal is at least ₹500, calculate:

Discount = Subtotal $\times$ 0.10

Otherwise:

Discount = 0

Step 11-

Calculate:

Final Amount = Subtotal + GST – Discount

Step 12-

Select the payment method.

Step 13-

Generate the bill.

Step 14-

Save the bill in canteen_bills.txt.

Step 15-

Reduce the purchased quantity from inventory.

Step 16-

If Admin is selected, authenticate the administrator.

Step 17-

Allow the administrator to manage inventory, prices, and food items.

Step 18-

Return to the main menu.

Step 19-

If Exit is selected, terminate the program.

Step 20-

Stop.

9.PSEUDOCODE :

BEGIN

DISPLAY Main Menu

REPEAT

DISPLAY:

1. Customer
2. Admin
3. Exit

INPUT choice

IF choice = 1 THEN

INPUT customer_name
CREATE empty order

REPEAT

DISPLAY Customer Menu
INPUT customer_choice

IF customer_choice = 1 THEN
DISPLAY Food Menu

ELSE IF customer_choice = 2 THEN
DISPLAY Categories

ELSE IF customer_choice = 3 THEN
INPUT search_name
SEARCH food item
DISPLAY result

ELSE IF customer_choice = 4 THEN
INPUT item_number
INPUT quantity

IF item exists AND stock is sufficient THEN

```
        ADD item to order
    ELSE
        DISPLAY error
    END IF

ELSE IF customer_choice = 5 THEN
    DISPLAY current order

ELSE IF customer_choice = 6 THEN
    REMOVE selected item

ELSE IF customer_choice = 7 THEN
    CLEAR order

ELSE IF customer_choice = 8 THEN

    CALCULATE subtotal
    CALCULATE GST

    IF subtotal >= 500 THEN
        APPLY 10% discount
    END IF

    CALCULATE final amount

    SELECT payment method

    GENERATE bill
    SAVE bill
    UPDATE inventory
    CLEAR order

ELSE IF customer_choice = 9 THEN
    EXIT Customer Menu

ELSE
    DISPLAY "Invalid choice"

END IF
```

```
        UNTIL customer_choice = 9

ELSE IF choice = 2 THEN

    INPUT admin_username
    INPUT admin_password

    IF credentials are valid THEN

        REPEAT

            DISPLAY Admin Menu
            INPUT admin_choice

            PERFORM selected admin operation

        UNTIL admin_choice = 6

    ELSE
        DISPLAY "Invalid login"
    END IF

ELSE IF choice = 3 THEN

    DISPLAY "Thank you"
    EXIT

ELSE

    DISPLAY "Invalid choice"

END IF

UNTIL choice = 3

END
```

10.PYTHON CODE

```
from datetime import datetime

|
|
CANTEEN_NAME = "SMART CANTEEN"

GST_RATE = 0.05

DISCOUNT_RATE = 0.10

DISCOUNT_LIMIT = 500

LOW_STOCK_LIMIT = 5

BILL_FILE = "canteen_bills.txt"

ADMIN_USERNAME = "admin"

ADMIN_PASSWORD = "1234"

|
|
menu = {

    1: {"name": "Veg Burger", "category": "Burger", "price": 60, "stock": 20},

    2: {"name": "Cheese Burger", "category": "Burger", "price": 80, "stock": 15},

    3: {"name": "Veg Pizza", "category": "Pizza", "price": 120, "stock": 12},

    4: {"name": "Paneer Pizza", "category": "Pizza", "price": 150, "stock": 10},

    5: {"name": "French Fries", "category": "Snacks", "price": 80, "stock": 25},

    6: {"name": "Veg Sandwich", "category": "Snacks", "price": 70, "stock": 18},

    7: {"name": "Samosa", "category": "Snacks", "price": 20, "stock": 40},

    8: {"name": "Masala Dosa", "category": "South Indian", "price": 90, "stock": 15},

    9: {"name": "Idli Sambar", "category": "South Indian", "price": 60, "stock": 20},

    10: {"name": "Veg Biryani", "category": "Main Course", "price": 110, "stock": 14},

    11: {"name": "Paneer Rice", "category": "Main Course", "price": 100, "stock": 12},

    12: {"name": "Cold Drink", "category": "Beverages", "price": 40, "stock": 30},
```

```

13: {"name": "Tea", "category": "Beverages", "price": 20, "stock": 50},
14: {"name": "Coffee", "category": "Beverages", "price": 30, "stock": 40},
15: {"name": "Ice Cream", "category": "Dessert", "price": 50, "stock": 20},
16: {"name": "Gulab Jamun", "category": "Dessert", "price": 45, "stock": 25},
}

|
order = {}

order_id = 1000

|
|
def line(char='=', size=68):

    print(char * size)

|
def title(text):

    line()

    print(text.center(68))

    line()

|
def pause():

    input("\nPress Enter to continue...")

|
def money(value):

    return f"₹{value:.2f}"

|
def get_next_order_id():

    global order_id

    order_id += 1

    return order_id

|
def calculate_subtotal():

```



```

    subtotal = 0

    for item_no, quantity in order.items():

        subtotal += menu[item_no]["price"] * quantity

    return subtotal

|
def calculate_bill_values():

    subtotal = calculate_subtotal()

    gst = subtotal * GST_RATE

    discount = subtotal * DISCOUNT_RATE if subtotal >= DISCOUNT_LIMIT else 0

    final_amount = subtotal + gst - discount

    return subtotal, gst, discount, final_amount

|
|
def display_menu():

    title("SMART CANTEEN FOOD MENU")

    print(f'{"No.':<5}{Item':<24}{Category':<18}{Price':>10}{Stock':>10}")

    line('-')

    for number, item in menu.items():

        print(f'{"number:<5}{item["name"]:<24}{item["category"]:<18}{money(item["price"]):>10}{item["stock"]:>10}")

    line('-')

|
def display_categories():

    title("FOOD CATEGORIES")

    categories = []

    for item in menu.values():

        if item["category"] not in categories:

            categories.append(item["category"])

```

```

    for index, category in enumerate(categories, 1):

        print(f"{index}. {category}")

|
def search_food():

    title("SEARCH FOOD")

    keyword = input("Enter item name or category: ").strip().lower()

    found = False

    for number, item in menu.items():

        if keyword in item['name'].lower() or keyword in item['category'].lower():

            print(f"{number}. {item['name']} - {money(item['price'])} - Stock: {item['stock']}")

            found = True

    if not found:

        print("No matching food item found.")

|
def add_item():

    display_menu()

    try:

        item_no = int(input("Enter item number: "))

        if item_no not in menu:

            print("Invalid item number.")

            return

        quantity = int(input("Enter quantity: "))

        if quantity <= 0:

            print("Quantity must be greater than zero.")

            return

        current = order.get(item_no, 0)

        if current + quantity > menu[item_no]["stock"]:

```

```

        print("Not enough stock available.")

    return

    order[item_no] = current + quantity

    print(f"{quantity} x {menu[item_no]['name']} added to order.")

except ValueError:

    print("Please enter valid numbers.")

|
def view_order():

    title("CURRENT ORDER")

    if not order:

        print("Your order is empty.")

    return

    print(f"{No.:<5}{Item':<25}{Qty':<8}{Amount':>12}")

    line('-')

    for item_no, quantity in order.items():

        item = menu[item_no]

        amount = item["price"] * quantity

        print(f"{item_no:<5}{item['name']:<25}{quantity:<8}{money(amount):>12}")

    line('-')

    print(f"Subtotal: {money(calculate_subtotal())}")

|
def remove_item():

    if not order:

        print("Your order is empty.")

    return

    view_order()

    try:

```

```

    item_no = int(input("Enter item number to remove: "))

    if item_no not in order:

        print("Item is not in your order.")

    return

    quantity = int(input("Enter quantity to remove: "))

    if quantity <= 0:

        print("Quantity must be positive.")

    elif quantity >= order[item_no]:

        del order[item_no]

        print("Item removed completely.")

    else:

        order[item_no] -= quantity

        print("Order quantity updated.")

    except ValueError:

        print("Please enter a valid number.")

|
def clear_order():

    if not order:

        print("Your order is already empty.")

    return

    confirm = input("Clear entire order? (y/n): ").lower()

    if confirm == 'y':

        order.clear()

        print("Order cleared.")

    else:

        print("Order was not cleared.")

|

```

```

def generate_bill(customer_name, payment_method='Not Paid'):

    if not order:

        print("Cannot generate a bill for an empty order.")

        return

    subtotal, gst, discount, final_amount = calculate_bill_values()

    bill_no = get_next_order_id()

    now = datetime.now().strftime('%d-%m-%Y %H:%M:%S')

    title('FINAL BILL')

    print(f"Order ID      : {bill_no}")

    print(f"Customer Name : {customer_name}")

    print(f>Date and Time : {now}")

    print(f"Payment Method : {payment_method}")

    line('-')

    print(f"{'Item':<25}{ 'Qty':<8}{ 'Rate':<12}{ 'Amount':>12}")

    line('-')

    for item_no, quantity in order.items():

        item = menu[item_no]

        amount = item["price"] * quantity

        print(f"{'item['name']':<25}{quantity:<8}{money(item['price']):<12}{money(amount):>12}")

    line('-')

    print(f"Subtotal      : {money(subtotal)}")

    print(f"GST (5%)       : {money(gst)}")

    print(f"Discount       : {money(discount)}")

    print(f"Final Amount   : {money(final_amount)}")

    line()

    print("Thank you for ordering from Smart Canteen!")

```

```

    return bill_no, final_amount
|
def choose_payment():
    title("PAYMENT METHOD")

    print('1. Cash')

    print('2. UPI')

    print('3. Card')

    while True:
        choice = input('Choose payment method: ').strip()

        if choice == '1':
            return 'Cash'

        if choice == '2':
            return 'UPI'

        if choice == '3':
            return 'Card'

        print("Invalid payment choice.")
|
def save_bill(customer_name, bill_no, payment_method):
    subtotal, gst, discount, final_amount = calculate_bill_values()

    now = datetime.now().strftime("%d-%m-%Y %H:%M:%S")

    with open(BILL_FILE, 'a', encoding='utf-8') as file:
        file.write('\n' + '=' * 68 + '\n')

        file.write('SMART CANTEEN BILL\n')

        file.write('=' * 68 + '\n')

        file.write(f'Order ID: {bill_no}\n')

        file.write(f'Customer: {customer_name}\n')

        file.write(f'Date: {now}\n')

```

```

        file.write(f'Payment: {payment_method}\n')

    for item_no, quantity in order.items():

        item = menu[item_no]

        amount = item["price"] * quantity

        file.write(f'{item["name"]} x {quantity} = ₹{amount:.2f}\n')

    file.write(f'Subtotal: ₹{subtotal:.2f}\n')

    file.write(f'GST: ₹{gst:.2f}\n')

    file.write(f'Discount: ₹{discount:.2f}\n')

    file.write(f'Final Amount: ₹{final_amount:.2f}\n')

    print(f'Bill saved to {BILL_FILE}.')

|
def checkout(customer_name):

    if not order:

        print("Your order is empty.")

        return

    payment = choose_payment()

    result = generate_bill(customer_name, payment)

    if result:

        bill_no, amount = result

        save_bill(customer_name, bill_no, payment)

        for item_no, quantity in order.items():

            menu[item_no]["stock"] -= quantity

        order.clear()

        print(f'Payment of {money(amount)} recorded successfully.')

|
|
|
def admin_login():

```

```

    title("ADMIN LOGIN")

    username = input("Username: ")

    password = input("Password: ")

    if username == ADMIN_USERNAME and password == ADMIN_PASSWORD:

        print("Admin login successful.")

        return True

    print("Invalid admin credentials.")

    return False

|
def show_inventory():

    title("INVENTORY")

    print(f'{"No.':<5}{Item':<25}{Stock':>10}{Status':>15}")

    line('-')

    for number, item in menu.items():

        status = "LOW STOCK" if item["stock"] <= LOW_STOCK_LIMIT else "Available"

        print(f'{"number:<5}{item['name']':<25}{item['stock']':>10}{status:>15}")

|
def restock_item():

    show_inventory()

    try:

        item_no = int(input("Enter item number: "))

        if item_no not in menu:

            print("Invalid item.")

            return

        quantity = int(input("Enter quantity to add: "))

        if quantity <= 0:

            print("Quantity must be positive.")

```



```

        return

    menu[item_no]["stock"] += quantity

    print("Stock updated successfully.")

except ValueError:

    print("Enter valid numbers.")

|
def update_price():

    display_menu()

    try:

        item_no = int(input("Enter item number: "))

        if item_no not in menu:

            print("Invalid item.")

            return

        price = float(input("Enter new price: "))

        if price <= 0:

            print("Price must be positive.")

            return

        menu[item_no]["price"] = price

        print("Price updated successfully.")

    except ValueError:

        print("Enter a valid price.")

|
def add_food_item():

    title("ADD FOOD ITEM")

    try:

        number = max(menu.keys()) + 1

        name = input("Food name: ").strip()

```

```

        category = input("Category: ").strip()

        price = float(input("Price: "))

        stock = int(input("Initial stock: "))

        if not name or not category or price <= 0 or stock < 0:

            print("Invalid food details.")

            return

        menu[number] = {"name": name, "category": category, "price": price, "stock": stock}

        print(f"Food item added with number {number}.")

    except ValueError:

        print("Enter valid price and stock.")

|
def remove_food_item():

    display_menu()

    try:

        number = int(input("Enter item number to delete: "))

        if number not in menu:

            print("Invalid item.")

            return

        print(f"Selected: {menu[number]['name']}")

        confirm = input("Delete this item? (y/n): ").lower()

        if confirm == 'y':

            del menu[number]

            print("Food item removed.")

        else:

            print("Deletion cancelled.")

    except ValueError:

        print("Enter a valid item number.")

```

```
|  
def admin_menu():  
    if not admin_login():  
        return  
    while True:  
        title("ADMIN MENU")  
        print('1. View Inventory')  
        print('2. Restock Item')  
        print('3. Update Price')  
        print('4. Add Food Item')  
        print('5. Remove Food Item')  
        print('6. Return')  
        choice = input('Enter choice: ').strip()  
        if choice == '1':  
            show_inventory()  
        elif choice == '2':  
            restock_item()  
        elif choice == '3':  
            update_price()  
        elif choice == '4':  
            add_food_item()  
        elif choice == '5':  
            remove_food_item()  
        elif choice == '6':  
            break  
        else:  
            print("Invalid choice.")
```

```
|  
|  
def help_menu():
```

```
    title("HELP")
```

```
    print('1. View the menu and choose an item number.')
```

```
    print('2. Add the required quantity.')
```

```
    print('3. View or modify your order.')
```

```
    print('4. Checkout and select a payment method.')
```

```
    print('5. The bill is automatically saved after checkout.')
```

```
|  
def about_project():
```

```
    title("ABOUT THE PROJECT")
```

```
    print('Project: Smart Canteen Ordering and Billing System')
```

```
    print('Language: Python')
```

```
    print('Level: First Year CSE Core')
```

```
    print('Concepts: Dictionaries, Functions, Loops, Conditions,')
```

```
    print('Exception Handling, File Handling and Datetime.')
```

```
|  
def customer_menu(customer_name):
```

```
    while True:
```

```
        title("CUSTOMER MAIN MENU")
```

```
        print('1. Display Food Menu')
```

```
        print('2. Search Food')
```

```
        print('3. Add Item')
```

```
        print('4. View Order')
```

```
        print('5. Remove Item')
```

```
        print('6. Clear Order')
```

```
        print('7. Checkout and Generate Bill')
```

```
print('8. Help')
```

```
print('9. About Project')
```

```
print('10. Return to Start Menu')
```

```
choice = input('Enter your choice: ').strip()
```

```
if choice == '1':
```

```
    display_menu()
```

```
elif choice == '2':
```

```
    search_food()
```

```
elif choice == '3':
```

```
    add_item()
```

```
elif choice == '4':
```

```
    view_order()
```

```
elif choice == '5':
```

```
    remove_item()
```

```
elif choice == '6':
```

```
    clear_order()
```

```
elif choice == '7':
```

```
    checkout(customer_name)
```

```
elif choice == '8':
```

```
    help_menu()
```

```
elif choice == '9':
```

```
    about_project()
```

```
elif choice == '10':
```

```
    break
```

```
else:
```

```
    print("Invalid choice. Please select 1-10.")
```

```
|
def start_program():

    while True:

        title("WELCOME TO SMART CANTEEN")

        print('1. Customer')

        print('2. Admin')

        print('3. Exit')

        choice = input('Enter your choice: ').strip()

        if choice == '1':

            customer_name = input("Enter customer name: ").strip()

            if customer_name:

                customer_menu(customer_name)

            else:

                print("Customer name cannot be empty.")

        elif choice == '2':

            admin_menu()

        elif choice == '3':

            print("Thank you for using Smart Canteen!")

            break

        else:

            print("Invalid choice.")

|
if __name__ == '__main__':

    start_program()
```

11.OUTPUT

```
=====
                                WELCOME TO SMART CANTEEN
=====

1. Customer
2. Admin
3. Exit
Enter your choice: 1
Enter customer name: Deepanwita Gera
=====

                                CUSTOMER MAIN MENU
=====

1. Display Food Menu
2. Search Food
3. Add Item
4. View Order
5. Remove Item
6. Clear Order
7. Checkout and Generate Bill
8. Help
9. About Project
10. Return to Start Menu
Enter your choice: 1
=====

                                SMART CANTEEN FOOD MENU
=====
```

No.	Item	Category	Price	Stock
1	Veg Burger	Burger	₹60.00	20
2	Cheese Burger	Burger	₹80.00	15
3	Veg Pizza	Pizza	₹120.00	12
4	Paneer Pizza	Pizza	₹150.00	10
5	French Fries	Snacks	₹80.00	25
6	Veg Sandwich	Snacks	₹70.00	18
7	Samosa	Snacks	₹20.00	40
8	Masala Dosa	South Indian	₹90.00	15
9	Idli Sambar	South Indian	₹60.00	20
10	Veg Biryani	Main Course	₹110.00	14
11	Paneer Rice	Main Course	₹100.00	12

10	Veg Biryani	Main Course	₹110.00	14
11	Paneer Rice	Main Course	₹100.00	12
12	Cold Drink	Beverages	₹40.00	30
13	Tea	Beverages	₹20.00	50
14	Coffee	Beverages	₹30.00	40
15	Ice Cream	Dessert	₹50.00	20
16	Gulab Jamun	Dessert	₹45.00	25

=====

CUSTOMER MAIN MENU

=====

1. Display Food Menu
 2. Search Food
 3. Add Item
 4. View Order
 5. Remove Item
 6. Clear Order
 7. Checkout and Generate Bill
 8. Help
 9. About Project
 10. Return to Start Menu
- Enter your choice: 2

=====

SEARCH FOOD

=====

Enter item name or category: Main Course

10. Veg Biryani - ₹110.00 - Stock: 14

11. Paneer Rice - ₹100.00 - Stock: 12

=====

CUSTOMER MAIN MENU

=====

1. Display Food Menu
2. Search Food
3. Add Item
4. View Order
5. Remove Item
6. Clear Order
7. Checkout and Generate Bill
8. Help
9. About Project

8. Help
9. About Project
10. Return to Start Menu
Enter your choice: 3

=====

SMART CANTEEN FOOD MENU

=====

No.	Item	Category	Price	Stock
1	Veg Burger	Burger	₹60.00	20
2	Cheese Burger	Burger	₹80.00	15
3	Veg Pizza	Pizza	₹120.00	12
4	Paneer Pizza	Pizza	₹150.00	10
5	French Fries	Snacks	₹80.00	25
6	Veg Sandwich	Snacks	₹70.00	18
7	Samosa	Snacks	₹20.00	40
8	Masala Dosa	South Indian	₹90.00	15
9	Idli Sambar	South Indian	₹60.00	20
10	Veg Biryani	Main Course	₹110.00	14
11	Paneer Rice	Main Course	₹100.00	12
12	Cold Drink	Beverages	₹40.00	30
13	Tea	Beverages	₹20.00	50
14	Coffee	Beverages	₹30.00	40
15	Ice Cream	Dessert	₹50.00	20
16	Gulab Jamun	Dessert	₹45.00	25

Enter item number: 10
Enter quantity: 6
6 x Veg Biryani added to order.

=====

CUSTOMER MAIN MENU

=====

1. Display Food Menu
2. Search Food
3. Add Item
4. View Order
5. Remove Item
6. Clear Order
7. Checkout and Generate Bill
8. Help

Problems Output Debug Console **Terminal** Ports

8. Help
9. About Project
10. Return to Start Menu
Enter your choice: 3

=====

SMART CANTEEN FOOD MENU

=====

No.	Item	Category	Price	Stock
1	Veg Burger	Burger	₹60.00	20
2	Cheese Burger	Burger	₹80.00	15
3	Veg Pizza	Pizza	₹120.00	12
4	Paneer Pizza	Pizza	₹150.00	10
5	French Fries	Snacks	₹80.00	25
6	Veg Sandwich	Snacks	₹70.00	18
7	Samosa	Snacks	₹20.00	40
8	Masala Dosa	South Indian	₹90.00	15
9	Idli Sambar	South Indian	₹60.00	20
10	Veg Biryani	Main Course	₹110.00	14
11	Paneer Rice	Main Course	₹100.00	12
12	Cold Drink	Beverages	₹40.00	30
13	Tea	Beverages	₹20.00	50
14	Coffee	Beverages	₹30.00	40
15	Ice Cream	Dessert	₹50.00	20
16	Gulab Jamun	Dessert	₹45.00	25

Enter item number: 8
Enter quantity: 5
5 x Masala Dosa added to order.

=====

CUSTOMER MAIN MENU

=====

1. Display Food Menu
2. Search Food
3. Add Item
4. View Order
5. Remove Item
6. Clear Order
7. Checkout and Generate Bill
8. Help

8. Help
9. About Project
10. Return to Start Menu
Enter your choice: 4

=====

CURRENT ORDER

=====

No.	Item	Qty	Amount
10	Veg Biryani	6	₹660.00
8	Masala Dosa	5	₹450.00

Subtotal: ₹1110.00

=====

CUSTOMER MAIN MENU

=====

1. Display Food Menu
2. Search Food
3. Add Item
4. View Order
5. Remove Item
6. Clear Order
7. Checkout and Generate Bill
8. Help
9. About Project
10. Return to Start Menu
Enter your choice: 5

=====

CURRENT ORDER

=====

No.	Item	Qty	Amount
10	Veg Biryani	6	₹660.00
8	Masala Dosa	5	₹450.00

Subtotal: ₹1110.00

Enter item number to remove: 10

Enter quantity to remove: 3

Order quantity updated.

=====

Enter item number to remove: 10
Enter quantity to remove: 3
Order quantity updated.

=====

CUSTOMER MAIN MENU

=====

1. Display Food Menu
 2. Search Food
 3. Add Item
 4. View Order
 5. Remove Item
 6. Clear Order
 7. Checkout and Generate Bill
 8. Help
 9. About Project
 10. Return to Start Menu
- Enter your choice: 7

=====

PAYMENT METHOD

=====

1. Cash
 2. UPI
 3. Card
- Choose payment method: 3

=====

FINAL BILL

=====

Order ID : 1001
Customer Name : Deepanwita Gera
Date and Time : 26-09-2026 16:49:23
Payment Method : Card

Item	Qty	Rate	Amount
Veg Biryani	3	₹110.00	₹330.00
Masala Dosa	5	₹90.00	₹450.00

Subtotal : ₹780.00
GST (5%) : ₹39.00
Discount : ₹78.00

Problems Output Debug Console Terminal Ports

Discount : ₹78.00
Final Amount : ₹741.00

=====

Thank you for ordering from Smart Canteen!
Bill saved to canteen_bills.txt.
Payment of ₹741.00 recorded successfully.

=====

CUSTOMER MAIN MENU

- =====
1. Display Food Menu
 2. Search Food
 3. Add Item
 4. View Order
 5. Remove Item
 6. Clear Order
 7. Checkout and Generate Bill
 8. Help
 9. About Project
 10. Return to Start Menu
- Enter your choice: 8

=====

HELP

- =====
1. View the menu and choose an item number.
 2. Add the required quantity.
 3. View or modify your order.
 4. Checkout and select a payment method.
 5. The bill is automatically saved after checkout.
- =====

CUSTOMER MAIN MENU

- =====
1. Display Food Menu
 2. Search Food
 3. Add Item
 4. View Order
 5. Remove Item
 6. Clear Order
 7. Checkout and Generate Bill
 8. Help
 9. About Project

9. About Project
10. Return to Start Menu
Enter your choice: 3

=====

SMART CANTEEN FOOD MENU

=====

No.	Item	Category	Price	Stock
1	Veg Burger	Burger	₹60.00	20
2	Cheese Burger	Burger	₹80.00	15
3	Veg Pizza	Pizza	₹120.00	12
4	Paneer Pizza	Pizza	₹150.00	10
5	French Fries	Snacks	₹80.00	25
6	Veg Sandwich	Snacks	₹70.00	18
7	Samosa	Snacks	₹20.00	40
8	Masala Dosa	South Indian	₹90.00	10
9	Idli Sambar	South Indian	₹60.00	20
10	Veg Biryani	Main Course	₹110.00	11
11	Paneer Rice	Main Course	₹100.00	12
12	Cold Drink	Beverages	₹40.00	30
13	Tea	Beverages	₹20.00	50
14	Coffee	Beverages	₹30.00	40
15	Ice Cream	Dessert	₹50.00	20
16	Gulab Jamun	Dessert	₹45.00	25

Enter item number: 9
Enter quantity: 2
2 x Idli Sambar added to order.

=====

CUSTOMER MAIN MENU

=====

1. Display Food Menu
2. Search Food
3. Add Item
4. View Order
5. Remove Item
6. Clear Order
7. Checkout and Generate Bill
8. Help

8. Help
9. About Project
10. Return to Start Menu
Enter your choice: 7

=====

PAYMENT METHOD

=====

1. Cash
2. UPI
3. Card
Choose payment method: 1

=====

FINAL BILL

=====

Order ID : 1002
Customer Name : Deepanwita Gera
Date and Time : 26-09-2026 16:49:43
Payment Method : Cash

Item	Qty	Rate	Amount
Idli Sambar	2	₹60.00	₹120.00

Subtotal : ₹120.00
GST (5%) : ₹6.00
Discount : ₹0.00
Final Amount : ₹126.00

=====

Thank you for ordering from Smart Canteen!
Bill saved to canteen_bills.txt.
Payment of ₹126.00 recorded successfully.

=====

CUSTOMER MAIN MENU

=====

1. Display Food Menu
2. Search Food
3. Add Item
4. View Order
5. Remove Item
6. Clear Order

```
6. Clear Order
7. Checkout and Generate Bill
8. Help
9. About Project
10. Return to Start Menu
Enter your choice: 9
```

```
=====
                        ABOUT THE PROJECT
=====
```

```
Project: Smart Canteen Ordering and Billing System
Language: Python
Level: First Year CSE Core
Concepts: Dictionaries, Functions, Loops, Conditions,
Exception Handling, File Handling and Datetime.
=====
```

```
=====
                        CUSTOMER MAIN MENU
=====
```

```
1. Display Food Menu
2. Search Food
3. Add Item
4. View Order
5. Remove Item
6. Clear Order
7. Checkout and Generate Bill
8. Help
9. About Project
10. Return to Start Menu
Enter your choice: 10
```

```
=====
                        WELCOME TO SMART CANTEEN
=====
```

```
1. Customer
2. Admin
3. Exit
Enter your choice: 2
```

```
=====
                        ADMIN LOGIN
=====
```

```
Username: admin
Password: 1234
```


Username: admin
Password: 1234
Admin login successful.

=====

ADMIN MENU

=====

- 1. View Inventory
 - 2. Restock Item
 - 3. Update Price
 - 4. Add Food Item
 - 5. Remove Food Item
 - 6. Return
- Enter choice: 1
- =====

=====

INVENTORY

=====

No.	Item	Stock	Status
1	Veg Burger	20	Available
2	Cheese Burger	15	Available
3	Veg Pizza	12	Available
4	Paneer Pizza	10	Available
5	French Fries	25	Available
6	Veg Sandwich	18	Available
7	Samosa	40	Available
8	Masala Dosa	10	Available
9	Idli Sambar	18	Available
10	Veg Biryani	11	Available
11	Paneer Rice	12	Available
12	Cold Drink	30	Available
13	Tea	50	Available
14	Coffee	40	Available
15	Ice Cream	20	Available
16	Gulab Jamun	25	Available

=====

=====

ADMIN MENU

=====

- 1. View Inventory

```
Problems Output Debug Console Terminal Ports

=====
ADMIN MENU
=====
1. View Inventory
2. Restock Item
3. Update Price
4. Add Food Item
5. Remove Food Item
6. Return
Enter choice: 2
=====

INVENTORY
=====
No. Item Stock Status
=====
1 Veg Burger 20 Available
2 Cheese Burger 15 Available
3 Veg Pizza 12 Available
4 Paneer Pizza 10 Available
5 French Fries 25 Available
6 Veg Sandwich 18 Available
7 Samosa 40 Available
8 Masala Dosa 10 Available
9 Idli Sambar 18 Available
10 Veg Biryani 11 Available
11 Paneer Rice 12 Available
12 Cold Drink 30 Available
13 Tea 50 Available
14 Coffee 40 Available
15 Ice Cream 20 Available
16 Gulab Jamun 25 Available
Enter item number: 2
Enter quantity to add: 5
Stock updated successfully.
=====

ADMIN MENU
=====
1. View Inventory
2. Restock Item
3. Update Price
```

- 3. Update Price
- 4. Add Food Item
- 5. Remove Food Item
- 6. Return

Enter choice: 3

SMART CANTEEN FOOD MENU

No.	Item	Category	Price	Stock
1	Veg Burger	Burger	₹60.00	20
2	Cheese Burger	Burger	₹80.00	20
3	Veg Pizza	Pizza	₹120.00	12
4	Paneer Pizza	Pizza	₹150.00	10
5	French Fries	Snacks	₹80.00	25
6	Veg Sandwich	Snacks	₹70.00	18
7	Samosa	Snacks	₹20.00	40
8	Masala Dosa	South Indian	₹90.00	10
9	Idli Sambar	South Indian	₹60.00	18
10	Veg Biryani	Main Course	₹110.00	11
11	Paneer Rice	Main Course	₹100.00	12
12	Cold Drink	Beverages	₹40.00	30
13	Tea	Beverages	₹20.00	50
14	Coffee	Beverages	₹30.00	40
15	Ice Cream	Dessert	₹50.00	20
16	Gulab Jamun	Dessert	₹45.00	25

Enter item number: 9

Enter new price: 120

Price updated successfully.

ADMIN MENU

- 1. View Inventory
- 2. Restock Item
- 3. Update Price
- 4. Add Food Item
- 5. Remove Food Item
- 6. Return

Enter choice: 4

ADD FOOD ITEM

Food name: french fries
Category: Snacks
Price: 120
Initial stock: 20
Food item added with number 17.

ADMIN MENU

1. View Inventory
 2. Restock Item
 3. Update Price
 4. Add Food Item
 5. Remove Food Item
 6. Return
- Enter choice: 5

SMART CANTEEN FOOD MENU

No.	Item	Category	Price	Stock
1	Veg Burger	Burger	₹60.00	20
2	Cheese Burger	Burger	₹80.00	20
3	Veg Pizza	Pizza	₹120.00	12
4	Paneer Pizza	Pizza	₹150.00	10
5	French Fries	Snacks	₹80.00	25
6	Veg Sandwich	Snacks	₹70.00	18
7	Samosa	Snacks	₹20.00	40
8	Masala Dosa	South Indian	₹90.00	10
9	Idli Sambar	South Indian	₹120.00	18
10	Veg Biryani	Main Course	₹110.00	11
11	Paneer Rice	Main Course	₹100.00	12
12	Cold Drink	Beverages	₹40.00	30
13	Tea	Beverages	₹20.00	50
14	Coffee	Beverages	₹30.00	40
15	Ice Cream	Dessert	₹50.00	20
16	Gulab Jamun	Dessert	₹45.00	25
17	french fries	Snacks	₹120.00	20

15	Ice Cream	Dessert	₹50.00	20
16	Gulab Jamun	Dessert	₹45.00	25
17	french fries	Snacks	₹120.00	20

Enter item number to delete: 14

Selected: Coffee

Delete this item? (y/n): y

Food item removed.

=====

ADMIN MENU

- =====
1. View Inventory
 2. Restock Item
 3. Update Price
 4. Add Food Item
 5. Remove Food Item
 6. Return

Enter choice: 6

=====

WELCOME TO SMART CANTEEN

- =====
1. Customer
 2. Admin
 3. Exit

Enter your choice: 3

Thank you for using Smart Canteen!

PS C:\Users\Deepanwita Gera\AppData\Local\Programs\Microsoft VS Code> █

12. PYTHON CONCEPTS USED :

The project demonstrates several important Python concepts.

Variables

Variables store values such as:

customer_name

quantity

price

subtotal

gst

discount

final_amount

Constants

Constants are used for fixed values:

GST_RATE = 0.05

DISCOUNT_RATE = 0.10

DISCOUNT_LIMIT = 500

Dictionaries

Dictionaries store menu and order information.

Functions

Functions divide the program into smaller modules.

Examples:

display_menu()

add_item()

view_order()

checkout()

generate_bill()

admin_menu()

Conditional Statements

if, elif, and else are used for decision-making.

Loops

while loops are used to keep menus running.

for loops are used to process food items and orders.

Exception Handling

try-except is used to handle invalid numeric input.

File Handling

The project uses file handling to save bills.

Date and Time

The datetime module is used to record the date and time of the bill.

13. TESTING

Testing is performed to check whether the program works correctly for different inputs.

Test Case	Input	Expected Result
1	Customer selected	Customer menu displayed
2	Valid food item	Item added
3	Invalid food item	Error message displayed
4	Quantity greater than stock	Order rejected
5	Valid quantity	Item added successfully
6	Subtotal below ₹500	No discount
7	Subtotal ₹500 or above	10% discount applied
8	Cash selected	Cash payment selected

9	UPI selected	UPI payment selected
10	Card selected	Card payment selected
11	Checkout	Bill generated
12	Successful checkout	Inventory reduced
13	Admin login valid	Admin menu displayed
14	Admin login invalid	Error displayed
15	Bill generated	Bill saved in text file

14. ADVANTAGES

1. Reduces manual billing work.
2. Reduces calculation errors.
3. Provides quick order processing.
4. Automatically calculates GST and discounts.
5. Maintains inventory information.
6. Provides multiple payment options.
7. Stores completed bills.
8. Provides admin management features.
9. Uses a simple and user-friendly menu.
10. Demonstrates practical Python programming concepts.
11. Easy to modify and expand.
12. Suitable for a small college canteen environment.

15. LIMITATIONS

1. The system is command-line based.
2. It does not have a graphical user interface.
3. It does not use a real database.
4. Payment is simulated and does not connect to real payment gateways.
5. Admin credentials are stored directly in the program.
6. The system does not support multiple users simultaneously.
7. Bills are stored in a text file rather than a database.
8. Inventory data is not permanently stored when the program is restarted.
9. There is no online food ordering functionality.
10. The project is designed primarily as an educational application.

16. FUTURE SCOPE

The project can be improved in the future by adding:

1. A graphical user interface using Tkinter or another GUI framework.
2. A MySQL or SQLite database.
3. Real UPI/payment gateway integration.
4. Customer registration and login.
5. Secure password encryption.
6. Online ordering functionality.
7. Digital receipt generation in PDF format.
8. QR-code based ordering.
9. Automatic low-stock notifications.
10. Sales reports and daily revenue analysis.
11. Food ratings and customer feedback.
12. Search and filtering by price or category.
13. Cloud-based storage.
14. Mobile application support.
15. Multiple-canteen or multiple-counter support.

17. CONCLUSION

The **Smart Canteen Ordering and Billing System using Python** provides a simple computerized solution for managing food orders, billing, payments, and inventory in a college canteen.

The project successfully demonstrates how Python programming can be applied to solve a practical real-world problem. Customers can view the menu, search for food, place orders, calculate bills, select payment methods, and receive generated bills. The administrator can manage inventory, prices, and food items.

The project also demonstrates important programming concepts such as

variables, dictionaries, functions, loops, conditional statements, exception handling, file handling, and date-time operations.

Although the current version is a basic command-line application, it provides a strong foundation for developing a more advanced canteen management system with a graphical interface, database, secure authentication, online ordering, and real payment integration.

